

Ingeniería y Calidad de Software

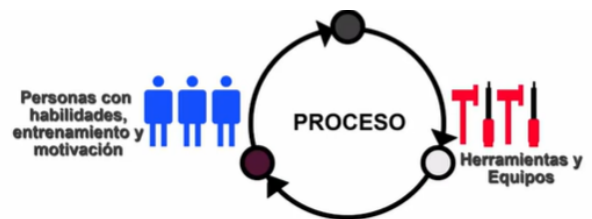
Componentes de proyecto de desarrollo de software

Proceso: transforma ideas y necesidades en un producto de software de calidad a nuestro cliente. Agrupa un conjunto de actividades, sumamos recursos y personas para obtener un resultado.

La secuencia de pasos ejecutados para un propósito dado (IEEE)

Proceso de Software: un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados

Las personas representan el 80% de nuestro proceso. Estas personas tienen que estar capacitadas, con conocimiento técnico y motivada.



Dentro de la industria existen dos tipos de procesos:

- Procesos **Empíricos**: tienen un fuerte arraigo en la experiencia, y la experiencia se genera.

Los procesos no están completos, son buenas prácticas, que las personas tienen que definir el proceso.

Se basa en ciclos cortos de implementación.

La experiencia no es extrapolable → no sirve como base para otros proyectos

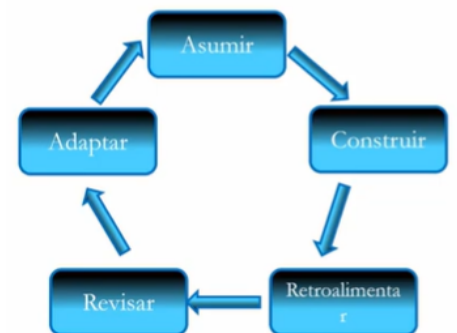
No se pueden combinar cualquier tipo de ciclo de vida → solo se puede usar el Iterativo/Incremental

- Procesos **Definidos**: asume que podemos repetir el mismo proceso una y otra vez indefinidamente, y obtener los mismos resultados.

Tiene una serie de pasos definidos de lo que se realiza.

La experiencia si es extrapolable.

Se pueden combinar cualquier tipo de ciclo de vida.



Ciclos de vida:

Serie de pasos a través de los cuales el producto o proyecto progresa.

Los productos tienen su ciclo de vida. Los proyectos también.

Un proceso elige un ciclo de vida. El ciclo de vida me dice como voy a encarar ese proceso, el orden y cuanto voy a hacer.

-En un ciclo de vida de un proyecto puede haber n ciclos de vida de un producto (versiones)

Un ciclo de vida de un producto inicia con la idea de ese producto y termina cuando lo retiró del mercado.



Un ciclo de vida de un proyecto de software es una representación de un proceso. Gráfica una descripción del proceso desde una perspectiva particular.

Tipos básicos de Ciclos de vida:

- Secuencial → cascada, versión entregable recién al final
- Iterativo/Incremental → pud
- Recursivo → espiral

Relación entre las 4P del desarrollo de software:

Proceso: conjunto de act. que toman como entrada requerimientos y tienen como salida un producto.

Proyecto: medio por el cual administro los recursos que necesitan, incorpora personas que asumen roles para la toma de decisiones.

Producto: software → conocimiento empaquetado a distintos niveles de abstracción. Un ítem de configuración es software.



Proyecto

- tienen un objetivo único → el resultado es distinto en cada proyecto → el producto tiene versiones eso es lo que lo hace diferente
- tienen temporalidad → tiene una fecha de inicio y fin
- los objetivos guían al proyecto → tiene que ser claro
→ tiene que ser alcanzable
- tareas interrelacionadas basadas en hechos? y recursos

→ Enfoque tradicional:

Administración/Gestión del proyecto: quiero que el proyecto cumpla con el objetivo, organizo las personas y los recursos, y doy un seguimiento para verificar si las cosas se están dando como era planeado.

- planificación
- seguimiento y control

Triple restricción:

- Alcance → producto/servicio y performance requerida (no lo manejamos nosotros)
- Tiempo → fecha comprometida (nosotros podemos determinarla)
- Costo → recursos y personas (nosotros podemos determinarla)

Producto Software: cada nueva versión es desarrollada incrementalmente es una serie de pasos.

Enfoque tradicional: líder de proyecto + equipo de proyecto. Estimar, planificar, asignar tareas y hacer un seguimiento.

Plan de proyecto: que, cuando, como y quien va a hacer el proyecto

La planificación implica:

- definición del alcance del proyecto
- definición de proceso y ciclo de vida
- estimación
- gestión de riesgos
- asignación de recursos
- programación de proyectos

- definición de controles
- definición de métricas

Alcance del Producto: son todas las características que pueden incluirse en un producto/servicio. → se mide en requerimientos y se definen en la ERS

Alcance del Proyecto: todo el trabajo que debe hacerse para entregar el producto/servicio con todas las características y funciones específicas. → se mide en tareas y se definen en el Plan de Proyecto

Definir un ciclo de vida : cualquier tipo de ciclo de vida

Estimaciones de software:

1. tamaño → del producto que voy a construir
2. esfuerzo → horas persona lineales
3. calendario → contesta cuando va a estar el producto
4. costo
5. recursos críticos → recursos raros (ej sensor)

Filosofía Ágil - Manifiesto Ágil

Compromiso útil entre nada de proceso y demasiado proceso (Fowler, 2001)

Basado en los Procesos Empíricos → con ciclos de realimentación cortos, que nos dan la información para revisar y corregir → el ciclo vuelve a empezar → ciclos de vida Iterativos

Tres pilares del empirismo:

- inspección
- adaptación
- transparencia: nos permite crecer como equipo, transformar el conocimiento explícito (nuestro) en conocimiento implícito para todo el equipo.

Valores Ágiles: ambas cosas son importantes pero son preferibles **estas**

- **Individuos e interacciones** por sobre Procesos y herramientas
- **Software funcionando** por sobre Documentación extensiva
- **Colaborar con el cliente** por sobre Negociación contractual
- **Responder al cambio** por sobre Seguir un plan

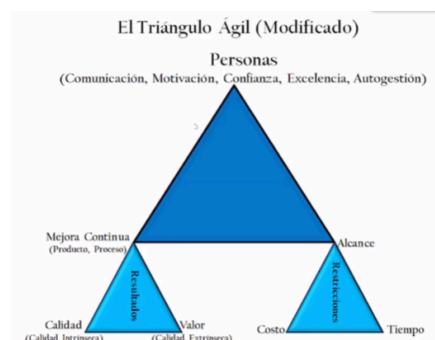
Principios: (doble click en los valores ágiles)

1. satisfacer al cliente
2. aceptar que los requisitos cambian
3. entregar software funcional frecuentemente
4. trabajar juntos
5. individuos motivados
6. conversaciones cara a cara
7. software funcionando
8. desarrollo sostenible
9. atención continua a la excelencia técnica y el buen diseño
10. la simplicidad es esencial
11. equipo auto-organizados
12. reflexión y ajuste del comportamiento

Alcance tradicional:



Alcance Ágil:

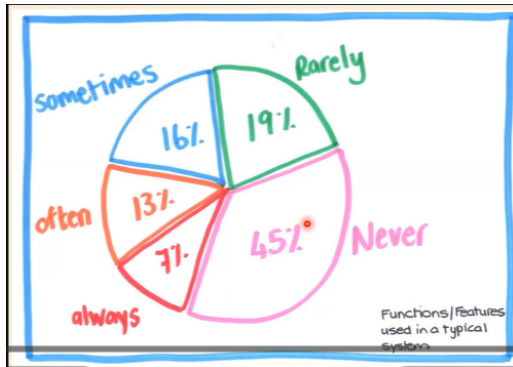


User Stories

Requerimientos en Agile:

- Usar el “valor” para construir el producto correcto
- Usar historias y modelos para mostrar que construir
- Determinar que es “solo lo suficiente”

El costo del tradicional BRUF:



→ **Gestión Ágil de Requerimientos de Software:** se define una prioridad en los requerimientos, para decidir que desarrollar primero.

Los requisitos cambiantes son una ventaja competitiva si se puede actuar sobre ellos.

just in time : analice cuando lo necesite, no antes

Tipos de Requerimientos: que se enfoca ágil

- de usuario
- de negocio → valor de negocio

Se define como fijo el tiempo y los recursos, dejando como estimado el alcance.

Principios ágiles relacionados a los Requerimientos Ágiles:

1. software funcionando
2. recibir cambios de requerimientos
3. técnicos y no técnicos trabajando juntos
4. comunicación cara a cara
5. equipos autoorganizados

User Stories

Descripción corta de una necesidad que tiene un usuario del software.

Partes de una User Story: (CCC)

- Tarjeta :
 - Frase verbal → descripción corta de la user
 - Como <nombre del rol> quien lo quiere
 - yo puedo <actividad> que quiero
 - de forma tal que <valor de negocio que recibo> para que → sirve para priorizar
 - Conversación
 - Confirmación:
 - Criterios de aceptación: define límites para una US (solo en la cátedra se implementó, en la formalidad de ágil no existe) - tienen que ser verificables - definen una intención, no una solución
 - Pruebas de aceptación: pruebas a partir de los criterios - descripción del caso de prueba.
- Probar <frase verbal> <escenario concreto> y <(pasa)/(falla)>

Detalles → se guardan en documentación interna del equipo. Ej: formato de número, lista desplegable.

Las historias son:

- una necesidad de usuario
- una descripción del producto
- un ítem de planificación
- token para una conversación
- mecanismo para diferir una conversación

Product Owner prioriza las historias en el Product Backlog

User stories: son Porciones Verticales para poder entregar valor al cliente → un poco de front - back - bd

Roles: tarjeta de Rol de Usuario → define al usuario

Usuarios representantes (Product Owner): gerente de usuarios/desarrollo, alguien de marketing, vendedores, expertos..

definición de listo → medida de calidad que el equipo define para identificar cuando una USB está lista para entrar a una iteración de desarrollo

de mínima tiene que tener de base : INVEST Model

I : independiente → calendarizable o implementable en cualquier orden

N : negociable → el “que” no el “cómo”

V : valuable → debe tener valor para el cliente

E : estimable → ayuda al cliente a armar un ranking basado en costos

S : small → en una iteración

T : testable → demostrar que fueron implementadas

Algo más de las User Stories:

- no son especificaciones detalladas de requerimientos
- son expresiones de intención
- no se detallan al principio del proyecto - se evita especificaciones anticipadas
- poco o nulo mantenimiento – se pueden descartar una vez implementadas
- sirven de entrada a la documentación

Niveles de abstracción:

US

Epica : US grande → no la pude ejecutar en una iteración → se dividen en más US

Tema : colección de user stories relacionadas

Spikes (CREO QUE NO VA)

Un tipo especial de US, se utiliza para disminuir la incertidumbre de una US.

Se clasifican en : técnicas y funcionales

Pueden utilizarse para:

- inversión básica para familiarizar al equipo con una nueva tecnología
- analizar comportamiento de una historia compleja y dividirla
- ganar confianza frente a riesgos tecnológicos
- interactuar con el usuario para resolver esos riesgos
- probar filtrar y ordenar planilla de gastos por

MVP - Mínimo Producto Viable

Conjunto mínimo de funcionalidades de un producto para validar una hipótesis.

- sirve para validar una hipótesis, para invertir lo mismo posible (tiempo-dinero-esfuerzo) y ver si el usuario está interesado en pagarlo.
- cuando tengo una idea de producto pero mucha incertidumbre.
- pensar en grande pero comienza de a poco
- puede ser un video, prototipo, etc

que queremos validar? → backlog - conjunto de funcionalidades

1. definir hipótesis
2. descripción/alcance
3. identificar US que incluye el MVP
4. fuera de alcance - US que no se incluyen

Estimaciones

- tienen un valor de probabilidad asociado
- no es precisa
- estimar no es planear y planear no es estimar

Técnicas fundamentales de estimación:

- Contar

Métodos utilizados:

- basados en la experiencia → datos históricos : proyección a partir de una bd históricos
→ juicio experto : puro (individual), delphi (grupal) - poker estimation
→ analogía
- basados exclusivamente en los recursos
- método basado exclusivamente en el mercado
- basados en los componentes del producto o en el proceso de desarrollo
- métodos algorítmicos

Estimaciones Ágiles

Lo beneficioso es el proceso.

Los features/stories son estimadas usando una medida de tamaño relativo → Story Points
las medidas relativas no son absolutas

- estimamos por comparación
- no es una medida basada en tiempo

El Story Point implica: complejidad-incertidumbre-esfuerzo

- cuán complejo es una story → complejidad
- cuanto trabajo es requerido para hacer o completar una story → esfuerzo
- cuan grande es una story
- cuánta incertidumbre técnica o de requerimiento/negocio hay → incertidumbre

Lo podemos medir como:

- tamaño por número: 1 a 10
- talles de remeras: S, M, L, XL, XXL
- serie 2n: 1, 2, 4, 8, 16, 32, 64, etc
- fibonacci: 0, 1, 2, 3, 5, 8, 13, etc

Story point: es una medida específica (del equipo) de complejidad, riesgo y esfuerzo, que nos permite hacer la comparación.

Poker estimation:

- combina la opinión de expertos con la desagregación de las US
- utiliza Fibonacci

Ponemos sobre la mesa la US que estamos por estimarla, charlamos de los desafíos, tareas de esa US y cada uno le asigna un puntaje. Si todos estimamos un mismo puntaje a esa US canónica → se elige la US más básica que nos va a servir como comparación para estimar las otras US

Si no todos estimamos con el mismo puntaje → se debate porque y se llega a un acuerdo
Al inicio de la sesión se define entre todos cuál va a ser nuestra US canónica

Velocidad: medida (métrica) del progreso de un equipo.

- Se calcula sumando el número de story points que el equipo completa durante la iteración.
- Se cuentan solo las completas, no parcialmente completas. Se realiza al final de una iteración.

Story Points totales / cant. de sprints = cant de SP por Sprint = Velocidad

duración del proyecto: SP totales / Velocidad = Cant total de sprints

SCRUM

Es un marco de trabajo liviano que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptativas para problemas complejos.

- genera valor
- requiere scrum master para fomentar un entorno
- marco que se puede emplear en diversos procesos, técnicas y métodos
- se basa en la inteligencia colectiva

Siento sus bases en → Empirismo → transparencia-inspección-adaptación

→ iterativo - incremental

→ Pensamiento Lean → filosofía para reducir los desperdicios

Valores de Scrum:

- compromiso
- enfoque → Sprint
- apertura hacia nuevos desafíos
- respeto
- coraje

Equipo Scrum:

responsable para crear un incremento valioso y útil en cada sprint

- equipos de no más de 10 personas
- multifuncional y autogestionada
- no posee jerarquía

→ Scrum Master ≠! Lider de proyecto

Responsable de establecer Scrum tal como define la guía. Sirve a: el equipo, product owner y organización

→ Product Owner

Responsable de maximizar el valor del producto resultante del trabajo del equipo de scrum

Responsable de la gestión eficaz de la pila del producto

Sus decisiones deben ser respetadas por la organización

Es una única persona

→ Developers

Quienes se comprometen a crear cualquier aspecto de un incremento útil en cada Sprint.

Responsable de:

- crear el sprint backlog
- inculcar la calidad adhiriéndose a una definición de hecho
- adaptar su plan hacia el objetivo sprint
- responsabilizarse mutuamente como profesionales

Ceremonias/Eventos de Scrum:

Timebox: tiempo fijo que tiene cada una de las ceremonias/evento del scrum

→ Sprint

Ciclos de trabajo, como proyectos cortos con un objetivo.

Al terminar un sprint, inmediatamente comienza uno nuevo.

Dentro de los sprints, se realizan las ceremonias.

→ Planificación del Sprint (Sprint Planning)

Se inicia el Sprint estableciendo el trabajo que se realizará para el mismo.

Es creado por el trabajo colaborativo de todo el equipo de Scrum.

- porque este sprint es valioso
- que se puede hacer en este sprint
- cómo se realizará el trabajo elegido

→ Daily Scrum

Reunión diaria. Evento de 15 minutos. Mismo tiempo. Mismo lugar. Facilitado por un scrum master.

- hay algún impedimento
- que hice ayer
- que hare hoy

→ Sprint Review

Inspecciona el resultado del Sprint, cuando se finaliza. Determina futuras actualizaciones.

Revisar, descubrir y reordenar la información.

→ Sprint Retrospective

Última actividad que se realiza al cerrar el sprint. Planificar formas de aumentar la calidad y eficiencia.

El equipo inspecciona como fue el último sprint y analiza: lo que salió bien, los problemas y cómo mejorar.

Artefactos de Scrum:

→ Product Backlog

Lista emergente y ordenada de lo que se necesita para mejorar el producto.

Es la única fuente del trabajo realizado por el Scrum Team.

Refinamiento (definición y división)

compromiso : objetivo del producto

→ Sprint Backlog

Compuesto por sprint goal, elementos del product backlog y un plan para el incremento.

Detallista para poder ver su progreso.

compromiso : sprint goal

- es el único objetivo del sprint
- crea coherencia y enfoque
- se puede negociar el alcance del sprint

responde a:

porque se hace el sprint → product owner

que vamos a realizar el sprint → product owner, scrum master y developers

cómo → developers

→ Product Increment

incremento: paso concreto hacia el objetivo, aditivo, debe proporcionar valor.

compromiso : definición de hecho (definition of done) → descripción del incremento, de acuerdo a los diferentes niveles user/incremento de producto/

- cuando cumple el incremento lo esperado
- controla el resultado de un sprint
- debe seguirse por todo el equipo y todos los equipos

Cada artefacto con su compromiso:

product backlog → objetivo del producto

sprint backlog → objetivo del sprint

product increment → DoD (definition of done)

Timebox en Scrum:

- Sprint : 1 mes o menos
- Sprint planning : 8 horas máximo para un Sprint de 1 mes
- Daily Meeting : 15 minutos
- Sprint Review : 4 horas máximo para un Sprint de 1 mes
- Sprint Retrospective : 3 horas máximo para un Sprint de 1 mes
- Refinamiento del PB : 10% del tiempo de Sprint - tiempo relativo porque no se sabe en qué momento exactamente se necesita hacer

Definition of Ready: criterio para determinar si entra al product backlog, como mínimo el modelo INVEST.

Definition of Done: criterio para determinar si entra al producto para que me lo acepte el product owner. calidad y características del producto

Capacidad del Equipo en un Sprint: → se calcula en el sprint planning
se puede estimar en tiempos de horas reales o de story points

Herramientas:

taskboard: código del item backlog, nombre de día del sprint, tiempo estimado de horas reales, iniciales de la persona.

configuración básica del tablero scrum:

1. story → estimadas en story points
 2. to do → tareas estimadas en horas ideales
 3. in process
 4. to verify (por el product owner)
 5. done
- tiene que tener una buena granularidad

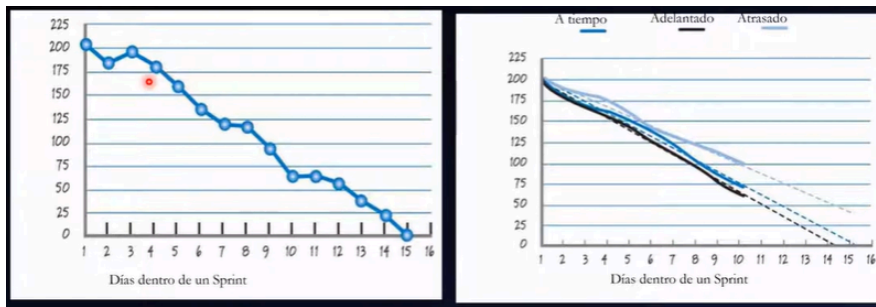
sprint burndown charts: → trabajo quemado

gráfico que permite visualizar el trabajo. eje vertical: story points, eje horizontal: días del sprint.

-se actualiza en la daily diciendo que está finalizado y que no.

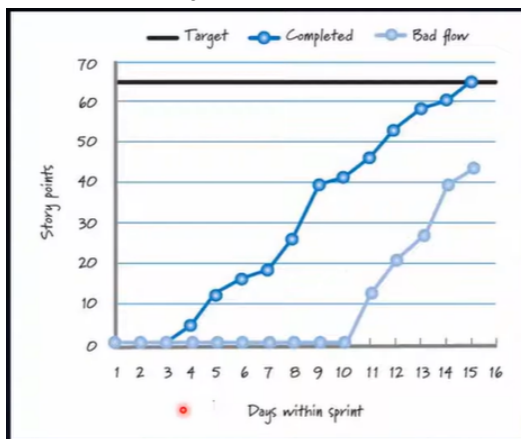
-se eliminan y crea otro al final de cada sprint - calculando la velocidad

-curva ideal vs curva real



sprint burnup chart:

- es fijo, es para todos los sprint
- muestra trabajo acumulado



→ cambios en el medio de un sprint no están permitidos ←
es preferible terminar el sprint y armar un nuevo sprint incluyendo lo que falta

Múltiples niveles de planificación: (del mas grande al más chiquito)

1. Estrategia →
2. Portfolio → conjunto de productos - a que producto le pongo mas o meno, cual discontinuo - portfolio backlog (S, M, L, XL)
3. Producto → sumatoria de los release a lo largo del tiempo
4. Release → versión de producto que se pone en producción - release planning
5. Iteración → sprint - sprint planning - sprint backlog
6. Día → daily

Nivel	Horizonte	Quién	Foco	Entregable
Portfolio	1 año o más	Stakeholders y Product Owners	Administración de un Portfolio de Producto	Backlog de Portfolio
Producto	Arriba de varios meses o más	Product Owner y Stakeholders	Visión y evolución del producto a través del tiempo	Visión de Producto, roadmap y características de alto nivel
Release	3 (o menos) a 9 meses	Equipo Scrum entero, Stakeholders	Balancear continuamente el valor de cliente y la calidad global con las restricciones de alcance, cronograma y presupuesto	Plan de Release
Iteración	Cada iteración (de 1 semana a 1 mes)	Equipo Scrum entero	Que aspectos entregar en el siguiente sprint	Objetivo del Sprint y Sprint Backlog
Día	Diaria	Scrum Master y Equipo de Desarrollo	Cómo completar lo comprometido	Inspección del progreso actual y adaptación a la mejor forma de organizar el siguiente día de trabajo

planificación y estimación se hace con respecto a ciertas circunstancias/condiciones, si algo cambia se tiene que volver a planificar y estimar

Métricas en ambientes ágiles:

- running tested features (RTS) : cuanta cant de características que estan funcionando que se desarrollaron en un sprint (story point) → se calcula
- capacidad : horas de trabajo disponible por día X días disponibles iteración → se estima
- velocidad : medida del progreso del equipo, se cuentan los story points de las US que están completas → se calcula

Cambios en las guías de scrum 2017-2020:

- aún menos perspectiva
- un equipo, centrado en un producto
- product goal → estado futuro - objetivo
- un espacio para el objetivo de sprint, la definición de hecho y el objetivo del producto
- autogestion en lugar de auto-organización
- tres temas en la planificación del sprint
- simplificación general del lenguaje para un público más amplio

SCM (Software Configuration Management)

Línea base:

Preguntas teóricas

- a. En base a lo descrito en el enunciado, ¿elegiría para gestionar el proyecto un proceso empírico o uno definido? Si elegimos un ciclo de vida iterativo e incremental, cuál sería la diferencia entre plantear una iteración en un proceso empírico y en un proceso definido. Justifique cada respuesta. (10 puntos)

Elegiría un proceso empírico porque existe incertidumbre, una fecha de entrega exigente y necesidad de adaptar el alcance. En un proceso empírico, cada iteración permite construir un incremento, inspeccionar y adaptar el trabajo siguiente. En un proceso definido, aun utilizando un ciclo iterativo-incremental, las iteraciones siguen actividades y procedimientos previamente definidos y el foco está en cumplir el proceso planificado.

- b. Suponiendo que elige la gestión tradicional de proyectos, ¿cómo ajustaría las variables de la triple restricción, en función de lo planteado por el presidente de la NBA? Justifique su respuesta. (10 puntos)

Ajustaría principalmente el alcance. El costo está restringido por el presupuesto de dos millones de dólares y el tiempo de la primera entrega está restringido por la fecha del Draft, el 27 de junio. Por lo tanto, debería priorizar las funcionalidades de mayor valor y postergar funcionalidades menos críticas. La segunda etapa puede negociarse en otra fecha, manteniendo el presupuesto acordado.

- c. Sabemos que el ciclo de vida de un proyecto de sw es diferente del ciclo de vida del producto de sw que se construye. ¿Cómo explicaría esto en el ejemplo concreto de este dominio? Justifique (10 puntos)

El proyecto es temporal, mientras que el producto puede continuar evolucionando durante años. En este caso, un proyecto puede ser desarrollar la primera versión del sistema para utilizarla en el Draft del 27 de junio. Una vez finalizado ese proyecto, el producto puede continuar utilizándose y nuevos proyectos pueden incorporar funcionalidades, cambios en las reglas del Draft o mejoras. El ciclo de vida del producto finalizará cuando la NBA deje de utilizarlo o lo retire.

- d. El MVP definido, ¿podría ser también el MMP? Justifique su respuesta. (10 puntos)

Sí, el MVP podría coincidir con el MMP, pero no necesariamente. El objetivo principal del MVP es validar una hipótesis con el mínimo esfuerzo posible, mientras que el MMP busca disponer del conjunto mínimo de características suficiente para liberar una versión útil del producto. Si el MVP definido para el Draft, además de permitir validar la hipótesis, tiene la calidad y funcionalidad necesarias para ser utilizado efectivamente por la NBA, podría ser también el MMP. De lo contrario, serían productos mínimos diferentes.

- e. ¿El modelo INVEST forma parte del criterio de READY? Haz tu propia definición del criterio de Ready para este ejercicio. (10 puntos)

Una User Story estará Ready para ingresar al Product Backlog cuando describa claramente una necesidad de un usuario, tenga un valor de negocio identificable, cumpla INVEST, sea suficientemente comprendida por el equipo, pueda estimarse y tenga condiciones verificables que permitan posteriormente confirmar su implementación.

Nro.	Pregunta	Escriba en esta columna las opciones de respuesta correcta
1.	Respecto a los ciclos de vida de proyectos de software, indique las afirmaciones correctas: A. Son una representación de un proceso B. Definen exclusivamente las técnicas de programación C. Especifican fases y su orden D. Son independientes del tipo de proyecto E. Influyen en la administración del proyecto	D X C E P
2.	Los criterios de aceptación sirven para: A. Definir cuándo una historia está completa B. Especificar la arquitectura C. Alinear al PO, testers y desarrolladores D. Documentar lo que no queda en ningún otro lugar de la US E. Reemplazar las conversaciones	B X C P
3.	Un PO exige convertir story points directamente a horas para comprometer fechas. ¿Qué problemas genera? A. Confusión entre tamaño y esfuerzo B. Uso indebido de las estimaciones C. Mayor precisión D. Presión indebida sobre el equipo E. Pérdida del enfoque empírico	A D B E 3p
	¿Cuáles diferencias correctas entre MVP, MMF y MLP? A. MVP se enfoca en aprendizaje B. MMF aporta valor comercial concreto C. MLP prioriza deleite emocional D. MLP elimina la necesidad de hipótesis E. MMF siempre es un prototipo	A B S C P
	En el marco de SCM, ¿cuáles afirmaciones describen correctamente una línea base (baseline)? A. Es una configuración revisada formalmente y acordada B. Puede cambiarse libremente si el equipo lo decide C. Permite "volver atrás" y reproducir el entorno en un punto del proyecto D. No debe confundirse con la versión del producto E. Es equivalente a "la rama principal (trunk/master)"	A C S D P

o.	Pregunta	las opciones de respuesta correcta
	Sobre Definition of Done (DoD) del material, ¿cuáles son típicamente parte del checklist? A. Código completo, refactorizado y con formato estándar B. Código en repositorio e inspeccionado ☒ C. Pruebas (unidad, integración, sistema) realizadas ☒ D. Cero defectos conocidos y despliegue en producción E. Tener "requerimientos detallados IEEE" completos	A C D 3p
	Durante una auditoría, se detecta que una funcionalidad implementada no cumple con lo especificado en los requerimientos aprobados, aunque el código compila y pasa tests técnicos. ¿Qué tipo de auditoría y conceptos están involucrados? ☒ A. Auditoría funcional de configuración (FCA) <i>WU → P. correcto → func</i> B. Auditoría física de configuración (PCA) <i>VEL → Confidencialmente → físicos</i> C. Verificación ☒ D. Validación E. Gestión de variantes	A D EX P
	En una Sprint Review, el PO rechaza varias historias porque "no era lo que esperaban", aunque el equipo afirma haberlas completado. ¿Qué problemas estructurales pueden haber ocurrido? ☒ A. Definition of Done poco clara o inexistente <i>si ya están siendo manejados dentro sprint</i> B. Falta de transparencia en el Sprint <i>deberían cumplir con DOR</i> C. Criterios de aceptación insuficientes ☒ D. Problemas en la Definition of Ready E. Uso incorrecto de la velocidad del equipo	A B C D 3p
	Un equipo elige un ciclo de vida secuencial para un sistema donde los usuarios finales recién podrán interactuar con el producto cerca del final del proyecto. ¿Qué consecuencias son coherentes con esta decisión? ☒ A. El feedback del usuario llega tarde ☒ B. El costo de corregir errores aumenta C. Se reduce el desperdicio de funcionalidades no usadas ☒ D. Se incrementa el riesgo de no satisfacer necesidades reales E. Se facilita la adaptación a cambios tardíos	A B D 5p
	En un proyecto ágil, el equipo observa que al repetir una iteración con las mismas prácticas obtienen resultados distintos en calidad y velocidad. Desde la perspectiva de procesos empíricos, ¿qué interpretaciones son correctas? ☒ A. Es esperable que los resultados varíen aun repitiendo el proceso ☒ B. La gestión debe basarse en inspección y adaptación C. El proceso empírico busca eliminar toda variabilidad ☒ D. La predictibilidad absoluta no es un supuesto válido E. El proceso empírico es inadecuado para trabajos complejos	A B D 5p

Pregunta	esta columna las opciones de respuesta correcta
¿Cuáles son razones válidas para no comparar directamente el desarrollo de software con la manufactura tradicional? A. El software no se gasta con el uso » B. El software se produce en masa C. No todas las fallas son errores D. El software está gobernado por leyes físicas E. El desarrollo depende fuertemente del factor humano »	A C E Sp
¿Cuáles son características esenciales de una User Story bien formada? A. Describe una necesidad del usuario B. Define detalladamente la solución técnica C. Tiene valor de negocio explícito D. Facilita la conversación E. Reemplaza a la ERS según estándar de la IEEE	A C E Sp
En estimaciones ágiles, los Story Points: A. Son una medida relativa » B. Representan tiempo C. Son específicos del equipo » D. Consideran complejidad, esfuerzo y duda » E. Permiten comparar equipos distintos	A C D Sp
Un MVP se caracteriza por: A. Maximizar aprendizaje validado B. Estar orientado a early adopters » C. Ser un producto comercial completo D. Minimizar esfuerzo inicial » E. Priorizar ganar mercado	A B D Sp
Un equipo lanza una feature completa sin validar hipótesis de valor. ¿Qué riesgo aumenta? A. Desperdicio » B. Time to market excesivo » C. Aprendizaje validado D. Construir algo que nadie quiere » E. Sobre-ingeniería	A B D E Sp

¿Cuáles asociaciones Artefacto → Compromiso son correctas? A. Product Backlog → Objetivo del Producto B. Sprint Backlog → Objetivo del Sprint C. Incremento del Producto → Definition of Done (DoD) D. Product Backlog → Definition of Ready (DoR) E. Incremento del Producto → Objetivo del Sprint	A B C Sp
Un proyecto "se atrasa de a un día por vez" y no hay seguimiento real del plan. ¿Qué causas de fracaso señaladas en el material encajan? A. Planificar basado en datos insuficientes B. No hay seguimiento del plan de proyecto C. Plan de proyecto pobre en detalles D. Estimaciones basadas en supuestos sin datos históricos E. Definir métricas mínimas y revisarlas frecuentemente	A B C D 3p
Un equipo trabaja en paralelo en dos ramas. Al integrar, aparecen conflictos y nadie puede asegurar qué cambios están incluidos en el ejecutable entregado al cliente. ¿Qué debilidades de SCM se evidencian en este escenario? A. Falta de identificación clara de ítems de configuración B. Ausencia de control de versiones efectivo C. Problemas de trazabilidad entre código fuente y ejecutable D. Uso incorrecto del concepto de variante E. Falta de auditorías de configuración	A B C E 2p
Un proyecto inicia con requisitos poco claros y con fuerte presión del negocio por comenzar rápido. La gerencia exige un plan detallado y fechas cerradas desde el día uno. Desde el enfoque visto en la materia, ¿qué riesgos o inconsistencias se identifican? A. Se está forzando un enfoque predictivo en un contexto de alta incertidumbre B. Se incrementa el riesgo de desvíos entre lo estimado y lo planificado C. Se favorece la inspección y adaptación continua D. Se asume que los requerimientos no cambiarán E. Se incrementa la probabilidad de entregar software que no satisface al cliente	A B E D 3p
Una empresa decide aplicar el mismo proceso de desarrollo que fue exitoso en un proyecto anterior, sin adaptarlo al nuevo equipo ni al nuevo dominio del problema. El proyecto fracasa. Según los conceptos iniciales de Ingeniería de Software, ¿qué afirmaciones explican mejor este resultado? A. El software es un producto altamente dependiente del factor humano B. Un proceso exitoso es necesariamente repetible en cualquier contexto C. El desarrollo de software es una actividad intelectual y creativa D. Los procesos definidos asumen predictibilidad que puede no existir E. El fracaso se debe exclusivamente a errores técnicos del equipo	A C D 2p

Nro.	Pregunta	Escriba en esta columna las opciones de respuesta correcta ↕
1.	Una startup ha invertido en una plataforma de desarrollo low-code esperando que sus desarrolladores junior puedan crear aplicaciones complejas sin experiencia técnica profunda. Como Líder de Proyecto, ¿qué afirmaciones serían coherentes con los planteamientos de Fred Brooks? Seleccione todas las opciones correctas: A) Las herramientas low-code pueden reducir dificultades accidentales como la codificación manual. B) Las dificultades esenciales, como la complejidad del dominio y la comunicación con el cliente, seguirán presentes. C) La experiencia técnica se vuelve irrelevante con herramientas modernas. D) Las herramientas pueden mejorar la productividad, pero no eliminar las dificultades esenciales.	A, B, D 5p
2.	Un proyecto de desarrollo de software para una entidad bancaria debe entregarse antes de lo previsto por una exigencia regulatoria. El cliente no está dispuesto a reducir el alcance ni aumentar el presupuesto. ¿Qué afirmaciones son consistentes con el planteo de la triple restricción? Seleccione todas las opciones correctas: A) Reducir el tiempo sin ajustar el alcance ni el costo, puede generar sobrecarga en el equipo. B) Para cumplir con el nuevo plazo, se debería reducir el alcance o aumentar el presupuesto. C) La calidad del producto podría verse afectada si se mantiene el alcance y el costo. D) El tiempo es una variable independiente que no afecta las otras restricciones.	A B C 4p
	Una empresa trabaja con sprints de dos semanas. El cliente solicita que se reduzca la duración de los sprints para acelerar la entrega. ¿Qué afirmaciones son coherentes con la gestión del tiempo en proyectos ágiles? Seleccione todas las opciones correctas: A) Reducir la duración del sprint puede limitar la cantidad de trabajo entregable. B) El tiempo en ágil es flexible y no afecta la calidad del producto. C) El tiempo en proyectos ágiles es fijo por iteración, pero puede ajustarse entre sprints si el equipo así lo decide. D) Acelerar los sprints sin ajustar el alcance puede generar deuda técnica.	A, C 2p

4.	En un proyecto ágil, el equipo decide reducir la documentación formal de requerimientos. ¿Qué prácticas justifican esta decisión según los principios ágiles? Selecciona todas las opciones correctas: A) La documentación detallada es obligatoria para cada iteración. B) Se prioriza la comunicación directa y continua con el cliente. C) Los requerimientos se documentan exhaustivamente antes de iniciar el desarrollo. D) Se utiliza el product backlog como herramienta viva para gestionar requerimientos. E) La documentación mínima viable permite mayor flexibilidad ante cambios	B, D <i>B, D</i>
5.	Utilizando el framework Scrum, ¿qué elementos pueden influir directamente en la planificación de un release? A) La velocidad del equipo en sprints anteriores. B) La definición de "Done" del equipo. C) La cantidad de bugs reportados en versiones anteriores (sin contexto). D) La capacidad estimada del equipo durante el release. E) La duración fija de cada sprint sin considerar el contenido.	A <i>B, D, Ap</i>
	En el contexto de estimaciones ágiles, ¿cuáles de las siguientes afirmaciones son verdaderas? A) Las estimaciones en puntos de historia permiten comparar esfuerzos relativos entre historias de usuario, sin necesidad de conocer su duración exacta. B) El uso de estimaciones temporales (horas/días) es preferido en metodologías ágiles por su precisión. C) Planning Poker fomenta la colaboración del equipo y reduce el sesgo individual en las estimaciones. D) Las estimaciones ágiles deben ser precisas para garantizar el cumplimiento del sprint. E) El valor de una historia de usuario puede influir en su estimación si se utiliza el método WSJF (Weighted Shortest Job First), "el trabajo más corto ponderado primero", cuyo objetivo es producir el máximo beneficio económico para una organización al aportar el máximo valor.	A y C <i>E, D</i>
7.	Durante una revisión de sprint, el equipo presenta los avances al Product Owner y stakeholders. Uno de los desarrolladores comenta que el Scrum Master debería haber intervenido más en la toma de decisiones técnicas. Por otro lado, el Product Owner propone reorganizar el equipo para acelerar la entrega de funcionalidades. ¿Cuáles de las siguientes afirmaciones reflejan correctamente los roles y responsabilidades en Scrum? A) El Scrum Master facilita el proceso Scrum, pero no toma decisiones técnicas ni de negocio. B) El Product Owner puede reorganizar el equipo si considera que eso mejora la entrega de valor. C) Los desarrolladores son los responsables de las decisiones técnicas y de cómo llevar a cabo el trabajo. D) El Scrum Master debe intervenir en las decisiones técnicas para asegurar la calidad del producto. E) El Product Owner define qué se construye, pero no cómo se construye.	A, C y E <i>Sp</i>

		correcta o
8.	Durante la planificación de un proyecto bajo gestión tradicional, el gerente de proyecto solicita un cronograma detallado con fechas de inicio y fin para cada tarea. El equipo técnico propone incluir márgenes de contingencia y documentar los supuestos. Sin embargo, algunos stakeholders presionan para acortar los plazos y eliminar tareas que consideran innecesarias. ¿Qué acciones están alineadas con buenas prácticas de gestión tradicional de proyectos en este contexto? A) Incluir márgenes de contingencia para mitigar riesgos y posibles desviaciones. B) Documentar los supuestos que afectan el cronograma para facilitar el análisis posterior. C) Eliminar tareas del cronograma si los stakeholders consideran que no aportan valor directo. D) Definir fechas de inicio y fin basadas en estimaciones realistas y recursos disponibles. E) Ajustar el cronograma únicamente en función de las expectativas del cliente.	A B y D 2p
9.	Durante la ejecución de un proyecto tradicional, el equipo técnico solicita modificar el alcance para incluir una funcionalidad adicional que no estaba prevista originalmente. El líder de proyecto evalúa el impacto en tiempo y costos, mientras que el cliente insiste en que se implemente sin alterar el cronograma. ¿Qué acciones están alineadas con buenas prácticas de gestión tradicional en este contexto? A) Evaluar el impacto del cambio en el alcance sobre el cronograma, costos y calidad. B) Registrar el cambio en el plan de proyecto solo si el cliente lo aprueba formalmente. C) Aplicar el proceso de gestión de cambios antes de incorporar la nueva funcionalidad. D) Implementar el cambio directamente si el equipo técnico lo considera beneficioso. E) Negociar con el cliente para ajustar el cronograma si el cambio es aceptado.	A y C E 2p
10.	Durante una auditoría de configuración funcional para un proyecto de desarrollo de software, el auditor detecta que varios ítems de configuración no tienen trazabilidad con los requisitos originales. El líder del proyecto argumenta que los cambios fueron menores y no requerían documentación formal. El equipo de calidad propone revisar el proceso de control de cambios. ¿Qué acciones están alineadas con buenas prácticas en auditorías de SCM? A) Aceptar cambios menores sin documentación si no afectan funcionalidades críticas. B) Revisar el proceso de control de cambios para asegurar que todos los ítems modificados estén registrados. C) Validar únicamente los ítems que fueron entregados al cliente final. D) Documentar las decisiones tomadas durante el desarrollo, incluso si los cambios parecen triviales. E) Verificar que todos los ítems de configuración tengan trazabilidad con los requisitos aprobados.	D B y E 2p